

Actualización de runtime.py a GUI (Tkinter)

1. Introducción y Contexto

El motor de ejecución de BrickScript (runtime.py) fue originalmente diseñado para entornos de consola (CLI), utilizando bibliotecas como `os` y `msvcrt` para el control de pantalla y la captura de entrada síncrona. La implementación actual migra la representación visual y el manejo de eventos a una **Interfaz Gráfica de Usuario (GUI)** utilizando la biblioteca **Tkinter**, la interfaz estándar de Python 2.7.

El objetivo de esta migración es mejorar la experiencia de usuario al ofrecer una representación visual **estable**, *flicker-free* y una gestión de **eventos asíncrona** (no bloqueante).

2. Componentes de Inicialización de la GUI

La clase `Juego.__init__` ha sido extendida para inicializar y configurar los *widgets* gráficos fundamentales.

2.1. Creación de la Ventana Principal

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Importaciones	<code>import Tkinter as tk</code>	Alias necesario para referenciar la librería principal de la GUI.
Raíz de la GUI	<code>self.root = tk.Tk()</code>	Crea la ventana principal del sistema operativo (el contenedor de nivel superior) que alojará todos los demás <i>widgets</i> .
Título	<code>self.root.title(...)</code>	Establece el texto que aparece en la barra de título de la ventana.

Cierre Seguro	<code>self.root.protocol("WM_DELETE_WINDOW", self.cerrar_ventana)</code>	Asigna la función <code>self.cerrar_ventana</code> para ser ejecutada cuando el usuario intenta cerrar la ventana a través del botón nativo 'X' (<code>WM_DELETE_WINDOW</code>). Esto garantiza que el <i>game loop</i> asíncrono se detenga limpiamente.
----------------------	--	--

2.2. Definición de las Áreas de Contenido

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Tamaño de Celda	<code>self.taman_celda = 25</code>	Define el tamaño en píxeles de cada unidad de la cuadrícula. Se utiliza para calcular las dimensiones del área de dibujo.
Área de Dibujo (Canvas)	<code>self.canvas = tk.Canvas(...)</code>	Crea un <i>widget</i> especializado para gráficos 2D . Reemplaza la impresión de caracteres de la consola. El Canvas se utiliza para dibujar los rectángulos de las piezas o la serpiente.
Marco de Puntuación	<code>self.marco_score = tk.Frame(...)</code>	Crea un contenedor (Frame) que organiza el contenido de la barra lateral (puntuación y controles).
Etiquetas de Texto	<code>self.label_score = tk.Label(...)</code>	Crea <i>widgets</i> de texto que se utilizan para mostrar la puntuación y los controles.
Empaquetado (Geometry)	<code>self.canvas.pack(...)</code>	Utiliza el gestor de geometría pack para colocar y organizar los <i>widgets</i> dentro de

		la ventana raíz, definiendo su posición relativa (ej. side=tk.LEFT).
--	--	--

3. Bucle Principal y Asincronía (Game Loop)

El bucle de juego basado en while True ha sido reemplazado por el mecanismo de programación de eventos de Tkinter.

3.1. Métodos de Control del Ciclo

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Inicio del Bucle	<code>self.root.after(50, self.game_loop)</code>	En <code>self.run()</code> , esta llamada programa la ejecución inicial de <code>self.game_loop</code> después de 50 milisegundos.
Ciclo del Juego	<code>def game_loop(self): ...</code>	La función central. Ejecuta la lógica de <code>ON_TICK</code> , la gravedad, y llama a <code>self.dibujar()</code> .
Reprogramación	<code>self.timer_id = self.root.after(50, self.game_loop)</code>	Al final de <code>game_loop</code> , se reprograma para volver a ejecutarse en 50 ms. Esto es una forma no bloqueante de crear un <i>frame rate</i> fijo, permitiendo que la GUI responda a otros eventos (como el teclado).
Ejecución Final	<code>self.root.mainloop()</code>	Inicia el bucle de eventos de Tkinter. Se mantiene ejecutando hasta que la ventana se cierra o se llama a <code>root.destroy()</code> . Reemplaza el bucle while infinito de la versión de consola.

4. Funciones de Dibujo (Renderización Gráfica)

La representación del juego se logra pintando formas en el *Canvas* en cada ciclo de juego.

4.1. Métodos de Renderización

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Limpieza de Pantalla	<code>self.canvas.delete("all")</code>	Se ejecuta al inicio de <code>self.dibujar()</code> . Borra todos los elementos dibujados previamente en el <i>Canvas</i> .
Dibujo de Celda	<code>def dibujar_celda(self, x, y, color):</code> ...	Función auxiliar que convierte las coordenadas lógicas de la cuadrícula (x, y) en coordenadas de píxeles de la pantalla (x1, y1, x2, y2) para el dibujo.
Dibujo de Rectángulo	<code>self.canvas.create_rectangle(...)</code>	Utiliza el método del <i>Canvas</i> para pintar un cuadrado sólido (un "brick") en la pantalla.
Actualización de Score	<code>self.label_score.config(text=...)</code>	Actualiza el texto del <i>widget</i> Label de la puntuación de forma dinámica en cada <i>frame</i> que se dibuja.

5. Manejo de Input y Salida Final

5.1. Manejo de Entrada

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Vinculación de Teclado	<code>self.root.bind('<Key>', self.manejar_input_gui)</code>	Vincula el evento de pulsación de cualquier tecla (<Key>) a la función de manejo de entrada.
Gestión de Input	<code>def manejar_input_gui(self, event): ...</code>	Recibe el objeto event de Tkinter, que contiene el símbolo de la tecla pulsada (<code>event.keysym.upper()</code>). Esto permite mapear teclas como 'UP' o 'DOWN' a las acciones de juego de forma no bloqueante.

5.2. Salida del Juego

Función / Atributo	Sintaxis	Propósito y Detalle Técnico
Mostrar Game Over	<code>tkMessageBox.showinfo(...)</code>	Muestra un cuadro de diálogo de mensaje nativo para el <i>Game Over</i> .
Cierre Completo	<code>self.root.destroy()</code> y <code>sys.exit(0)</code>	Se asegura de que la ventana gráfica se cierre y que el proceso de Python termine por completo después de que el usuario haga clic en "Aceptar" en el cuadro de diálogo de <i>Game Over</i> .